

五一数学建模竞赛

承 诺 书

我们仔细阅读了五一数学建模竞赛的竞赛规则。

我们完全明白，在竞赛开始后参赛队员不能以任何方式（包括电话、电子邮件、网上咨询等）与本队以外的任何人（包括指导教师）研究、讨论与赛题有关的问题。

我们知道，抄袭别人的成果是违反竞赛规则的，如果引用别人的成果或其它公开的资料（包括网上查到的资料），必须按照规定的参考文献的表述方式在正文引用处和参考文献中明确列出。

我们郑重承诺，严格遵守竞赛规则，以保证竞赛的公正、公平性。如有违反竞赛规则的行为，我们愿意承担由此引起的一切后果。

我们授权五一数学建模竞赛组委会，可将我们的论文以任何形式进行公开展示（包括进行网上公示，在书籍、期刊和其他媒体进行正式或非正式发表等）。

参赛题号（从 A/B/C 中选择一项填写）：_____ C _____

参赛队号：_____ T3837726888284 _____

参赛组别（研究生、本科、专科、高中）：_____ 本科 _____

所属学校（学校全称）：_____ 西南交通大学 _____

参赛队员： 队员 1 姓名：_____ 黄 璨 _____

队员 2 姓名：_____ 王子杨 _____

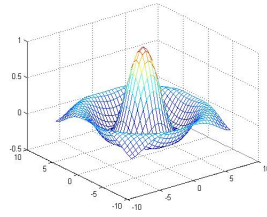
队员 3 姓名：_____ 周启珩 _____

联系方式： Email: _____ 1527093952@qq.com _____ 联系电话： _____ 18884243437 _____

日期： _____ 2024 年 5 月 1 日 _____

（除本页外不允许出现学校及个人信息）

五一数学建模竞赛



题目：煤矿深部开采冲击地压危险预测

关键词：

摘要：

问题一中，我们首先需要对干扰数据进行数据探索，找出干扰数据的显著特征，我们通过对一定时间内的干扰数据进行小波分析之后，发现干扰数据具有显著的变化和一定的周期性，除此之外，干扰数据在开始阶段与结束阶段有较大的变化，于是我们通过小波分析： $f(t) = \sum_{j=1}^{\infty} \sum_{k \in \mathbb{Z}} c_{j,k} \varphi(2^{j-1}(t-k)) + \sum_{k \in \mathbb{Z}} d_k \varepsilon(t-k)$ ，将所需要的特征进行提取，进行初步的判断之后，选取了平均值、方差、原始数据方差等数据进行具体值的计算，随后利用随机森林模型对这些特征进行判断，最终得到干扰数据的六个特征：电磁辐射（声波强度）、最大值、时间、滚动最大值、滚动最小值、滚动平均值。随后问题一要求我们构造模型对附件2的数据进行判断，并找到干扰数据最早发生的五个时间段，对此我们使用随机森林模型对干扰数据进行预测，将数据分为干扰数据与非干扰数据，由此得到的第一个电磁辐射干扰数据时间段为：2022-05-02 02:56:18——2022-05-02 11:32:04，第一个声发射干扰数据时间段为：2022-04-03 09:27:51——2022-04-03 10:00:46

问题二中，我们需要对前兆数据进行数据探索，找出其特征。我们同样按照问题一的思路，对前兆数据进行小波分析： $f(t) = \sum_{j=1}^{\infty} \sum_{k \in \mathbb{Z}} c_{j,k} \varphi(2^{j-1}(t-k)) + \sum_{k \in \mathbb{Z}} d_k \varepsilon(t-k)$ ，此问中首先统计前兆数据，发现处于150-300之间的数据不属于前兆数据，为其中一个显著的特征，随后，我们通过随机森林模型，判断其特征，得到了以下特征：时间、星期、平均值、最大值、最小值、滚动平均值，滚动最大值，滚动最小值。随后问题二要求我们对前兆数据进行预测，我们同样使用随机森林模型，其中主要为熵公式： $P(X = x_i) = p_i, i = 1, 2, \dots, n$ ， $H(X) = -\sum_{i=1}^n p_i \log(p_i)$ ，以及基尼不纯度公式： $Gini(p) = \sum_{k=1}^K p_k(1-p_k) = 1 - \sum_{k=1}^K p_k^2$ ，由此得到的第一个电磁辐射前兆数据时间段为：2020-04-09 14:25:02——2020-04-09 21:57:15，第一个声发射干扰数据时间段为：2021-11-01 18:10:30——2021-11-01 18:14:08

问题三中，我们需要对前兆数据的概率进行预测，找出其规律。我们同样在问题二的基础上，加入时间间隔这个特征量来考虑其数据的非连续性，并搭建bp神经网络深度学习来推出概率值。我们导入特征量作为输入层，并选用ReLU (Rectified Linear Unit) 函数： $f(x) = \max(0, x)$ 作为激活函数，并在输出层引入 Softmax 函数： $\sigma(t) = \frac{1}{1+e^{-t}}$

来进行概率归一化，并与交叉熵损失函数： $D_{KL}(p||q) = \sum_{i=1}^N p(x_i) \log(\frac{p(x_i)}{q(x_i)})$ 搭配，提高数值稳定性，从而构建了一个双线性的神经网络模型来预测概率

一、 问题重述

1.1 问题背景

煤炭是中国的主要能源和重要工业原料。然而，随着开采深度的增加，地应力增大，井下煤炭动力灾害风险越来越大，严重影响着煤炭的安全高效开采。在各类深部煤岩动力灾害事故中，冲击地压已成为威胁中国煤矿安全生产最重要的灾害之一，冲击地压事故易造成严重的人员伤亡和财产损失。近年来，研究人员进行了大量深入的研究，采取了许多防控措施，中国煤矿安全形势持续稳步改善。但是，冲击地压事故仍时有发生，煤矿安全形势依然严峻，冲击地压的监测预警和有效防控仍是煤矿安全生产中仍待解决的科技问题。

在深部煤矿开采过程中，可以监测声发射（AE）和电磁辐射（EMR）信号，电磁辐射和声发射传感器每 30 秒采集一个数据，可通过这些数据的变化趋势判断目前工作面或巷道是否存在冲击地压危险。电磁辐射和声发射数据随着采煤工作面的推进波动，一般在冲击地压发生前数天（0-7 天，即大约冲击地压发生前 7 天内）会有一些趋势性前兆特征。

1.2 问题提出

问题一：针对传感器的干扰数据，请构建模型至少找出电磁辐射以及声波干扰数据的三个显著特征。随后构建模型，利用之前得到的显著特征对附件 2 中的数据进行预测，区分其中的干扰数据。

问题二：针对传感器的前兆数据，请构造模型至少找出电磁辐射以及声波前兆数据的三个重要特征，以此将其与其他数据区分。随后利用模型对附件 2 中的数据进行预测，找出其中的前兆数据。

问题三：针对传感器的数据，请构造模型找到寻找数据特征以找到每个时间点之后发生前兆数据的可能性，由此提前预测冲击地压，并预测给定时间点电磁辐射与声波数据之后发生前兆数据的可能性。

二、 问题分析

问题一：该问分为两小问，其中第一小问要求我们至少找出 3 个电磁辐射和声发射的干扰信号数据的特征。从数据分析上，我们首先可以研究所有种类信号的时间、平均值、方差、滚动最大值、滚动最小值与滚动平均值等，随后找出 C 类信号（干扰信号）与其他数据的显著差别，随后我们也可以对信号进行小波分析，研究小波分析之后的各项特征，由此得出更深刻的结论。如下为小波分析前 C 类信号电磁辐射的图像。

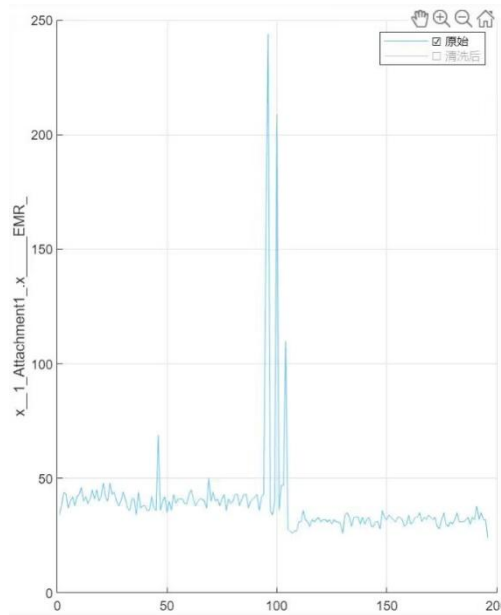


图 1 小波分析前 C 类信号电磁辐射数据

随后将 D/E 类数据、A、B、C 类数据的声发射和电磁辐射信号分别按照时间作出曲线，观察其变化情况，并根据变化情况初步确定特征值，随后我们通过小波分析将信号分解成一系列小波函数的叠加，这些小波函数都是具有有限支撑集并在正负之间振荡的波形。通过伸缩和平移运算，小波变换能够实现对信号的多尺度聚焦分析，从而精准地提取出有用信息。提取出大量特征后，还需要进行特征选择，以保留与任务最相关的特征。此时我们考虑选择随机森林模型进行多分类处理，提取出特征。

对于声发射的干扰我们也使用类似的方法进行分析。

如下为 C 类信号声发射的图像。

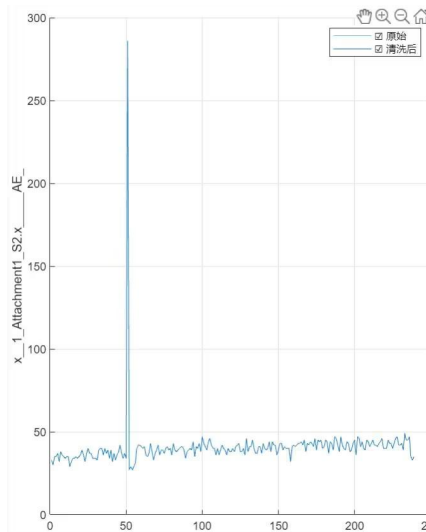


图 2 小波分析前 C 信号声发射数据

该问题的第二小问中要求我们建立数学模型找到最早的 5 个干扰信号，其实际只需要将我们第一小问中构建的特征指标代入其中即可，随后采用机器学习模型进行训练即可，这里我们选择使用随机森林模型，将附件 2 中的数据进行处理提取特征值，再通过随机森林进行分类，得到结果。

问题二：问题二与问题一大致相同，但问题二的要求变为探索前兆数据即 B 类数据的特征，我们采用的方法也与问题一相类似，同样分析其平均值、方差、极差等，如下为 B 类信号电磁辐射的图像。

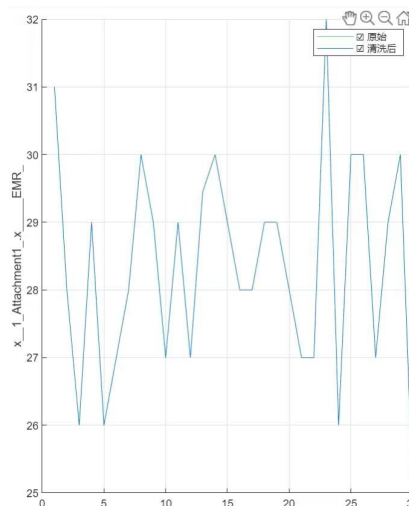


图 3 小波分析前 B 类信号电磁辐射数据
对于声发射的干扰我们也使用类似的方法进行分析。

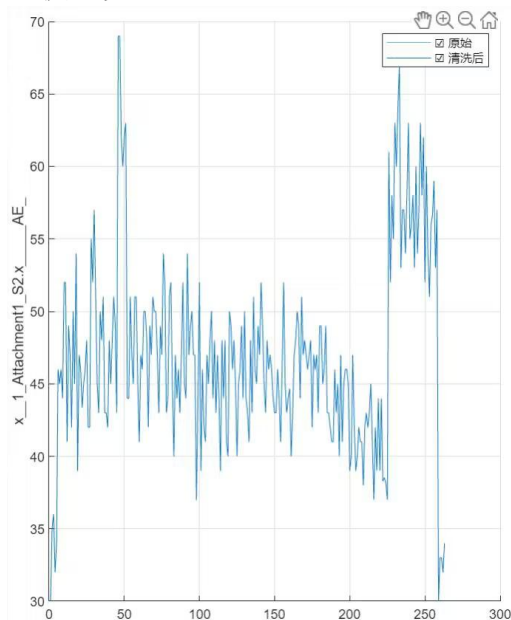


图 4 小波分析前 B 类信号声发射数据
如下为 B 类数据的频方图：

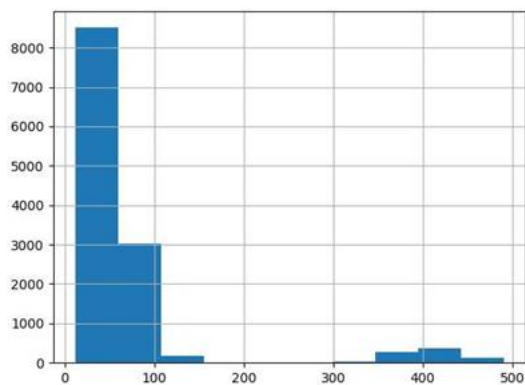


图 5 B 类数据频方图

通过观察分析我们不难发现，前兆数据大致分为两类，一类是低频辐射即电磁辐射小于 150 的，另外一类是高频辐射即电磁辐射大于 300 的，所以我们可以认为处于 150—300 之间的电磁辐射数据不属于前兆数据。

对于第二小问，我们采取与问题一中第二小问相同的方法进行计算，即通过小波分析处理数据之后，再使用随机森林模型提取出特征。

问题三：对于问题三，由于题中所给的电磁辐射和声发射信号数据是非连续的，本文选择在原有的特征上引入时间间隔这个特征量，并构建 bp 神经网络输出概率值

问题三同样运用小波分析对数据进行处理降噪后，对特征提取后的基因数据采用 BP 神经网络进行分类学习，训练获得分类网络模型，采用获得的分类网络模型对测试数进行分类，并对神经网络分类器的性能进行评估。

神经网络模型的构建，本文沿用对附件 1 小波提取的特征值作为数据集，构建了一个 BPNN 由输入层、输出层以及一个或多个隐含层组成。本文采用单隐含层的 三层神经网络拓扑结构，图。输入层神经元数目和样本数相同，隐含 神经元个数一般通过实验或根据经验值选取，输出层神经元个数为 1。通过神经网络算法与附件三中数据比较计算各时间节点的发生冲击地压的概率。

三、 模型假设与符号说明

3.1 模型基本假设

- (1) 传感器的数据准确且数据标注在整体上是正确的。
- (2) 各类数据的持续时间是连续的，且在其发生的过程中存在一定的规律。
- (3) 数据来源的煤矿并没有发生显著差别。
- (4) 传感器收集的数据来源相同。

3.2 符号说明

表 1 符号说明

符号	含义	单位
$a_{(t)}$	电磁辐射系数数据	
$b_{(t)}$	声发射系数数据	
x_1	电磁辐射能量	
x_2	声发射的能量	
x_3	电磁辐射的平均值	
x_4	声发射的平均值	
x_5	电磁辐射的标准差	
x_6	声发射的标准差	
x_7	电磁辐射的最大值	
x_8	声发射的最大值	
x_9	电磁辐射的最小值	
x_{10}	声发射的最小值	
x_{11}	电磁辐射的中位数	
x_{12}	声发射的中位数	
x_{13}	电磁辐射的峰值	

x_{14}	声发射的峰值
x_{15}	电磁辐射的总能量
x_{16}	声发射的总能量
x_{17}	电磁辐射能量比率
x_{18}	声射能量比率

四、数据预处理

4.1 数据清洗

实际应用中，信号往往会被各种噪声所污染。这些噪声可能来自于环境干扰、测量设备的误差、传输过程中的失真等等。滤波的主要目的之一就是通过特定的算法或硬件电路，从原始信号中提取出有用信息，同时抑制或去除噪声，从而提高信号的信噪比（SNR）。

本文采用卡尔曼滤波法建立模型，卡尔曼滤波法的原理主要是基于线性系统状态方程，通过系统的输入输出观测数据，对系统状态进行最优估计的算法。具体来说，它利用线性系统状态方程，通过系统输入输出观测数据，对系统状态进行最优估计。由于观测数据中包括系统中的噪声和干扰的影响，所以最优估计也可看作是滤波过程。

如下为使用 `matlab` 对数据进行处理之后的结果，分别得到电磁辐射和声发射信号的滤波结果。

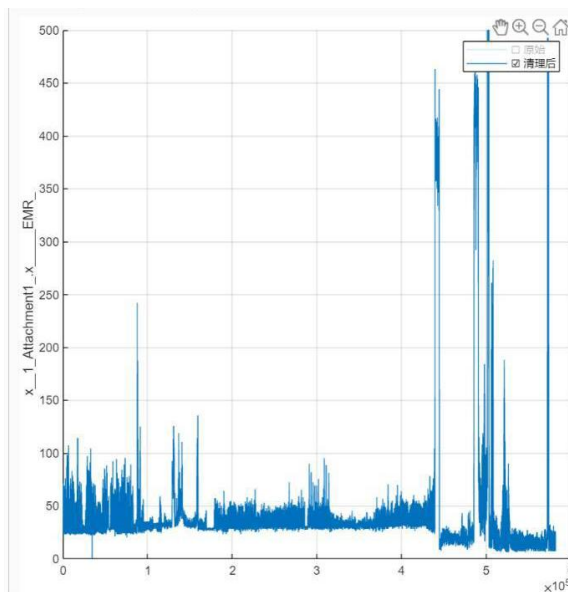


图 6 电磁辐射信号的滤波结果

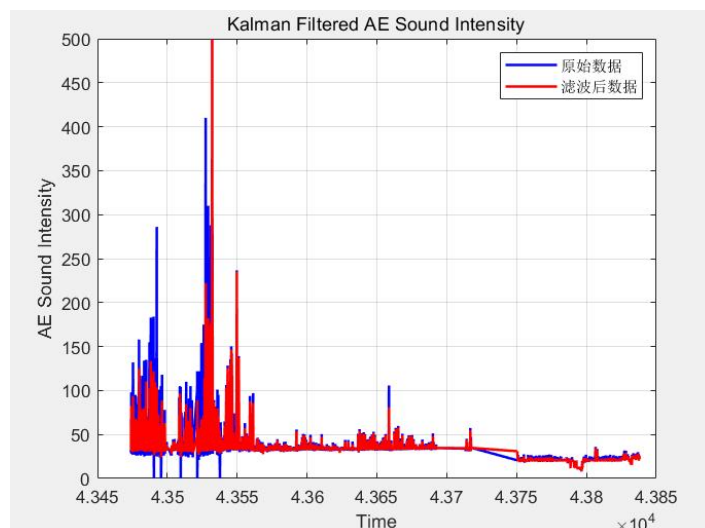


图 7 声发射信号的滤波结果

4.2 数据预处理

4.2.1 问题一、二中数据预处理

分别将 A、B、C、D/E 类信号对于时间画出折线图，观察两图中各条曲线的电磁辐射与声发射信号关于时间的变化情况。

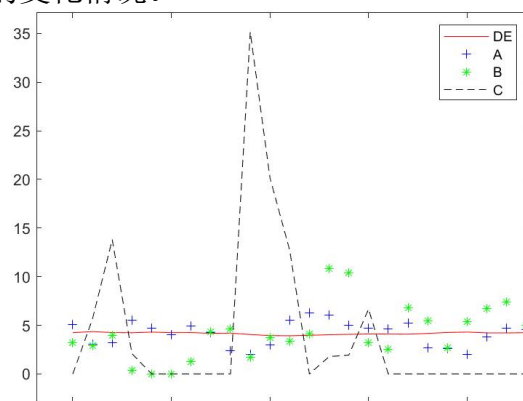


图 8 声发射信号关于时间的变化

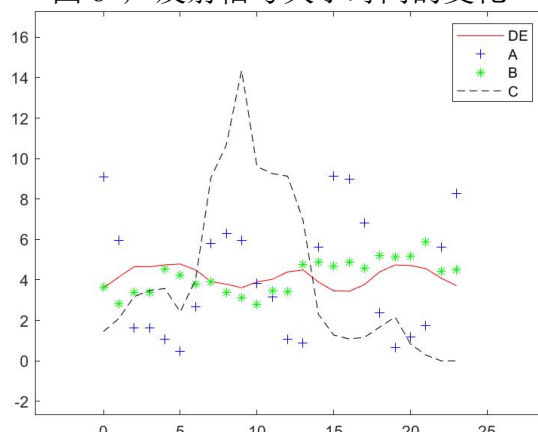


图 9 电磁辐射信号关于时间的变化

通过观察不难发现，两图中 C 类信号图像随着时间的变化而产生的变化幅度较大，即电磁辐射和声发射信号随着时间的变化较大，那么我们可以知道，时间为其中一个特征值。

对数据进行统计，分别得到 AE、EMR 与时间的对应关系以及分别计算 AE、EMR 的平均值、方差、最大值与最小值。

由此得到下列图像，从左至右依次为 D/E、A、B、C 类信号关于声发射信号的相关计算值。

count	592816.000000	count	15587.000000	count	2212.000000	count	1289.000000
mean	32.901326	mean	37.432989	mean	41.668658	mean	59.969106
std	13.186831	std	3.727368	std	8.303728	std	80.421827
min	0.000000	min	29.000000	min	29.000000	min	27.000000
25%	29.000000	25%	35.290000	25%	35.410000	25%	35.000000
50%	33.419000	50%	36.661000	50%	39.105000	50%	42.376000
75%	34.867000	75%	38.265500	75%	45.163750	75%	52.195000
max	310.000000	max	80.000000	max	81.000000	max	500.000000

以及下列图像，从左至右依次为 D/E、A、B、C 类信号关于电磁辐射信号的相关计算值。

count	519723.000000	count	73418.000000	count	12532.000000	count	5259.000000
mean	42.088690	mean	49.815421	mean	71.054808	mean	77.958504
std	62.663233	std	18.279324	std	91.899844	std	90.715876
min	0.000000	min	9.610000	min	11.670000	min	0.000000
25%	27.890000	25%	42.451000	25%	30.148750	25%	31.620000
50%	31.000000	50%	46.489500	50%	41.000000	50%	45.000000
75%	37.000000	75%	52.000000	75%	71.674000	75%	75.000000
max	500.000000	max	270.000000	max	491.000000	max	500.000000

五、模型建立与求解

5.1 问题一模型建立与求解

5.1.1 问题一求解思路

问题一要求我们建立模型分析电磁辐射信号以及声发射信号中干扰数据的特征，我们首先对一定周期内的数据采用小波分析，随后分析干扰信号的均值、方差以及其对应的值等。最后我们可以发现在小波分析之后，高饶信号相比其他类型的信号存在着更大的变化趋势，且在一定时段内呈现一定的周期性，但仍然存在较大的波动，所以我们依次为特征建立除了干扰信号的特征识别模型。随后我们将之前提取得到的指标加入到随机森林模型中进行训练，由此得到对应的判别方法

5.1.2 问题一模型建立

小波分析提取特征值的原理主要基于小波变换。我们将信号分解成一系列小波函数的叠加，这些小波函数都是具有有限支撑集并在正负之间振荡的波形。通过伸缩和平移运算，小波变换能够实现对信号的多尺度聚焦分析，从而精确地提取出有用信息。

小波变换^[1]能够将信号分解成不同频率范围的分量，信号在不同频率尺度上的分布情况可以通过不同分解尺度上信号的能量体现出来。为了说明各重构子频带能量与原信号能量的关系，对小波分解后各子频带的能量分布进行研究。设信号的离散序列为 $f(n)$ ，利用小波变换对其进行 J 尺度的分解与重构，得到 $J+1$ 个频率区间的分量，即

$$f(n) = A_j f(n) + D_j f(n) + D_{j-1} f(n) + \dots D_1 f(n)$$

式中： A_j 为低频部分小波分解系数； D_j 为高频部分小波分解系数， $j = 1, 2, \dots, J$ 。

在信号总能量不变的情况下，有

$$\begin{cases} E_j^A = \sum_{n=1}^N [A_j f(n)]^2 \\ E_j^D = \sum_{n=1}^N [D_j f(n)]^2 \end{cases}$$

式中： E_j^A 为信号在分解尺度J 上的低频信号分量的 能量； E_j^D 为信号在分解尺度 j 上的高频信号分量的 能量。 信号的总能量为

$$E = E_j^A + \sum_{j=1}^J E_j^D$$

小波分解的各个尺度对应的是信号中不同频率 范围内的分量，设， P_j^A, P_j^D 分别表示低频信号和高频 信号的小波能量百分比，则

$$\begin{cases} P_j^A = \frac{E_j^A}{E} \times 100\% \\ P_j^D = \frac{E_j^D}{E} \times 100\% \end{cases}$$

小波分析提取特征值的过程可以分为几个步骤：

① 信号分解：首先，通过小波变换将数据分解成不同尺度的小波系数。这些小波系数表示了信号在不同频率和不同时间点上的特性。

首先，傅里叶展开式：

$$f(t) = a_0 + \sum_{n=1}^{\infty} (a_n \cos(n\omega t) + b_n \sin(n\omega t)) \quad (1)$$

通过式子(1)得到小波基展开式：

$$f(t) = \sum_{k \in \mathbb{Z}} c_k \varepsilon(2^j t - k) \quad (2)$$

在此式子中，j 起到缩放函数的作用，类似于傅里叶变换中 ω ，k 是平移因子。

由于平移因子的作用，小波变换可以对局部细节进行分析，并且窗口大小是变化的。而傅里叶比那换针对整个函数（周期）进行分析。

小波变换分解表达式：

$$f(t) = \sum_{j=1}^{\infty} \sum_{k \in \mathbb{Z}} c_{j,k} \varphi(2^{j-1}(t - k)) + \sum_{k \in \mathbb{Z}} d_k \varepsilon(t - k) \quad (3)$$

其中 $\varepsilon(t)$ 为母小波， $\varphi(t)$ 为父小波，且母小波和父小波正交。

母小波性质：

平移正交性： $\langle \varepsilon(t), \varepsilon(t - kT) \rangle = 0$

归一性： $\langle \varepsilon(t), \varepsilon(t) \rangle = \int_{-\infty}^{\infty} \varepsilon^2(t) dt = 1$

父小波性质：

平移正交性： $\langle \varphi(t), \varphi(t - kT) \rangle = 0$

归一性： $\langle \varphi(t), \varphi(t) \rangle = \int_{-\infty}^{\infty} \varphi^2(t) dt = 1$

小波基的选取

足够的消失矩 $\int_{\mathbb{R}} t^k \varepsilon(t) dt = 0, 0 \leq k \leq m$

消失矩阶数的大小反映信号微小突变的检测能量。

② 特征提取：通过分析这些小波系数，可以提取出信号的各种特征。本文选择了以下波的基本特征：

通过判断之后，我们选取了以下值进行小波分析，提取特征值。

计算各尺度的能量 (scale_energies):

$$x_1 = a_{(t)}^2 \quad (4)$$

$$x_2 = b_{(t)}^2 \quad (5)$$

对每个尺度的系数（假设 coefficients 的第二个维度代表尺度），计算其绝对值的平方和。

计算各尺度的平均值 (scale_means):

$$x_3 = \frac{\sum |a(t)|}{t} \quad (6)$$

$$x_4 = \frac{\sum |b(t)|}{t} \quad (7)$$

对每个尺度的系数，计算其绝对值的平均值。

计算各尺度的标准差 (scale_stds):

$$x_5 = \frac{\sqrt{\sum [a(t)-u]^2}}{t} \quad (8)$$

$$x_6 = \frac{\sqrt{\sum [b(t)-u]^2}}{t} \quad (9)$$

对每个尺度的系数，计算其绝对值的标准差。

计算各尺度的最大值 (scale_max):

$$x_7 = \max a(t) \quad (10)$$

$$x_8 = \max b(t) \quad (11)$$

对每个尺度的系数，找到其绝对值的最大值。

计算各尺度的最小值 (scale_min):

$$x_9 = \min a(t) \quad (12)$$

$$x_{10} = \min b(t) \quad (13)$$

对每个尺度的系数，找到其绝对值的最小值。

计算各尺度的中位数 (scale_median):

$$x_{11} = \text{median } a(t) \quad (14)$$

$$x_{12} = \text{median } b(t) \quad (15)$$

对每个尺度的系数，计算其绝对值的中位数。

计算各尺度的峰值到峰值 (scale_ptp):

$$x_{13} = \max a(t) - \min a(t) \quad (16)$$

$$x_{14} = \max b(t) - \min b(t) \quad (17)$$

对每个尺度的系数，计算其绝对值的最大值与最小值的差（即峰值到峰值）。

计算总能量 (total_energy):

$$x_{15} = \sum a_{(t)}^2 + 1e-10 \quad (18)$$

$$x_{16} = \sum b_{(t)}^2 + 1e-10 \quad (19)$$

对所有尺度的系数，计算其绝对值的平方和作为总能量，为了避免除以零的情况或防止由于数值计算误差导致的极端情况,本文选择在总能量后加上 $1e-10$ ，它小到通常不会对总能量产生显著影响，但足以防止由于数值问题导致的除零错误。

计算各尺度能量的比率 (scale_energy_ratios):

$$x_{17} = \frac{a_{(t)}^2}{E} \quad (20)$$

$$x_{18} = \frac{b_{(t)}^2}{E} \quad (21)$$

对每个尺度的系数，计算其绝对值的平方与总能量 total_energy 的比率。

此外，我们还创建时间基特征，通过在一定大小的时间窗口或数据点上滑动，并对每个窗口内的数据进行统计计算，在这里本文选择窗口大小为 10，同时 10 个数据对应的时间，星期，平均值，最大值，最小值，加入以下特征：滚动平均值，滚动最大值，滚动最小值。

特征选择：在提取出大量特征后，还需要进行特征选择，以保留与任务最相关的特征。随后建立随机森林模型进行多分类处理，得到最终的特征。

随机森林^[2]是测变量的组合，因此每棵树都依赖于随机向量独立采样，森林中所有树木的分布相同。概括当森林中的树数变大时，森林的误差会收敛到极限。概括树木分类器森林的误差取决于森林中单个树木的强度和它们之间的关系。使用随机选择的特征来拆分每个节点会产生比较的错误率，但在噪声方面更稳健。内部估计监控误差，强度和相关性，这些用于显示对增加中使用的特征数量的响应分裂。内部估计也用于衡量变量重要性。这些想法也适用于回归。

得到以下结果：

	Feature Name	Selected	Ranking
0	电磁辐射 (EMR)	True	1
4	max	True	1
11	hour	True	1
13	rolling_mean	True	1
14	rolling_max	True	1
15	rolling_min	True	1
1	energy	False	2
12	day_of_week	False	3
10	energy_ratio_scale_3	False	4
2	mean	False	5
8	energy_ratio_scale_1	False	6
3	std	False	7
6	median	False	8
5	min	False	9
9	energy_ratio_scale_2	False	10
7	ptp	False	11

图 10 声发射信号的随机森林结果

	Feature Name	Selected	Ranking
0	电磁辐射 (EMR)	True	1
4	max	True	1
11	hour	True	1
13	rolling_mean	True	1
14	rolling_max	True	1
15	rolling_min	True	1
1	energy	False	2
12	day_of_week	False	3
10	energy_ratio_scale_3	False	4
2	mean	False	5
8	energy_ratio_scale_1	False	6
3	std	False	7
6	median	False	8
5	min	False	9
9	energy_ratio_scale_2	False	10
7	ptp	False	11

图 11 电磁辐射信号的随机森林结果

由此可以得到特征为电磁辐射（声波强度）、最大值、时间、滚动最大值、滚动最小值、滚动平均值

5.1.3 问题一模型求解与分析

通过训练集的导入学习，对测试集的信号类型进行预测，与该信号的原类型对比，得到预测的准确度，可推断该模型的精确度良好，在此基础上得到下列两图：

	precision	recall	f1-score	support
0	0.88	0.33	0.48	4694
1	0.89	0.25	0.39	668
2	0.98	0.42	0.59	418
3	0.98	1.00	0.99	177792
accuracy			0.98	183572
macro avg	0.93	0.50	0.61	183572
weighted avg	0.98	0.98	0.97	183572

图 12 电磁辐射的精度

	precision	recall	f1-score	support
0	0.81	0.67	0.73	22216
1	0.95	0.47	0.63	3748
2	0.91	0.32	0.47	1591
3	0.94	0.98	0.96	155725
accuracy			0.93	183280
macro avg	0.90	0.61	0.70	183280
weighted avg	0.92	0.93	0.92	183280

图 13 声发射信号的精度

A、B、C 分别对应为 0、1、2，那么从表中就可以看出 C 类信号的精度分别为 0.98 和 0.91，由此可见该模型对此预测有较好的结果。

将附件 2 的数据进行数据处理提取特征值，再通过随机森林进行分类，最终得到以下结果：

表 2 电磁辐射干扰信号时间区间

序号	时间区间起点	时间区间终点
1	2022-05-02 02:56:18	2022-05-02 11:32:04
2	2022-05-10 06:40:47	2022-05-10 13:37:42
3	2022-05-13 07:13:23	2022-05-13 07:31:41
4	2022-05-16 15:00:55	2022-05-16 17:56:31
5	2022-05-19 05:25:15	2022-05-19 07:31:28

表 3 声发射干扰信号时间区间

序号	时间区间起点	时间区间终点
1	2022-04-03 09:27:51	2022-04-03 10:00:46
2	2022-04-26 09:56:59	2022-04-26 12:37:55
3	2022-05-03 07:41:00	2022-05-03 07:48:19
4	2022-05-29 11:34:44	2022-05-29 12:38:49
5	2022-10-31 13:02:23	2022-10-31 13:11:31

5.2 问题二模型建立与求解

5.2.1 问题二求解思路

问题二要求我们建立模型分析电磁信号以及声发射中前兆数据的特征。该题总体思路与第一问相同，但前兆数据相比干扰数据更为稳定，周期性更强，所以我们可以从数据的分布等角度进行更多层次的分析找出更多的规律。

第二问同样进行数据清洗，小波分析提取特征值，但在特征值的提取中加入滚动窗口创建时间基特征，通过在一定大小的时间窗口或数据点上滑动，并对每个窗口内的数据进行统计计算，在这里本文选择窗口大小为 10，同时 10 个数据对应的时间，星期，平均值，最大值，最小值，加入以下特征：滚动平均值，滚动最大值，滚动最小值，同时在随机森林分类时，把 B 类信号归为 1 类，其它信号归为 0 类。

5.2.2 问题二模型建立

我们在该题中仍然采用小波分析对问题进行分析，并用随机森林进行求解，其中，小波分析的建模如下：

$$f(t) = \sum_{j=1}^{\infty} \sum_{k \in \mathbb{Z}} c_{j,k} \varphi(2^{j-1}(t-k)) + \sum_{k \in \mathbb{Z}} d_k \varepsilon(t-k) \quad (22)$$

通过式(22)进行小波分析后，提取出信号的各种特征，随后通过建立随机森林模型进行分类处理。

对于随机森林模型，主要由以下两个模型构成：

基尼系数：分类问题中，假设有 K 个类，样本点属于第 k 类的概率为 p_k ，则概率分布的基尼指数定义为：

$$Gini(p) = \sum_{k=1}^K p_k (1 - p_k) = 1 - \sum_{k=1}^K p_k^2 \quad (23)$$

熵 (entropy) 是表示随机变量不确定性的度量。设有随机变量 X，则：

$$P(X = x_i) = p_i, i = 1, 2, \dots, n \quad (24)$$

则随机变量 X 的熵定义为：

$$H(X) = - \sum_{i=1}^n p_i \log(p_i) \quad (25)$$

熵只依赖 X 的分布，而与 X 的取值无关，所以也可将 X 的熵记为 $H(X)$

当随机变量 X 只取(0,1)即服从 Bernoulli Distribution 则 X 的分布为：

$$P(X = 1) = p, P(X = 0) = 1 - p \quad (26)$$

熵为：

$$H(X) = -p \log_2 p - (1 - p) \log_2 (1 - p) \quad (27)$$

设有随机变量(X,Y)联合分布概率为

$$P(X = x_i, Y = y_j) = p_{ij}, i = 1, 2, \dots, n, j = 1, 2, \dots, m \quad (28)$$

由此可得以下两图：

	Feature Name	Selected	Ranking
0	电磁辐射 (EMR)	True	1
4	max	True	1
11	hour	True	1
13	rolling_mean	True	1
14	rolling_max	True	1
15	rolling_min	True	1
1	energy	False	2
12	day_of_week	False	3
10	energy_ratio_scale_3	False	4
2	mean	False	5
8	energy_ratio_scale_1	False	6
3	std	False	7
6	median	False	8
5	min	False	9
9	energy_ratio_scale_2	False	10
7	ptp	False	11

图 14 随机森林下声发射的特征

	Feature Name	Selected	Ranking
0	电磁辐射 (EMR)	True	1
4	max	True	1
11	hour	True	1
13	rolling_mean	True	1
14	rolling_max	True	1
15	rolling_min	True	1
1	energy	False	2
12	day_of_week	False	3
10	energy_ratio_scale_3	False	4
2	mean	False	5
8	energy_ratio_scale_1	False	6
3	std	False	7
6	median	False	8
5	min	False	9
9	energy_ratio_scale_2	False	10
7	ptp	False	11

图 15 随机森林下电磁辐射的特征

得出 10 个数据对应的时间、星期、平均值、最大值、最小值，加入以下特征：滚动平均值，滚动最大值，滚动最小值。

5.2.3 问题二模型求解与分析

我们通过 python 求解上述模型，再通过随机森林求解各项数据的准确度，得到以下两个图表：

	precision	recall	f1-score	support
0	1.00	1.00	1.00	182904
1	0.91	0.26	0.41	668
accuracy			1.00	183572
macro avg	0.95	0.63	0.70	183572
weighted avg	1.00	1.00	1.00	183572

图 16 B 类信号的电磁辐射精确度

	precision	recall	f1-score	support
0	0.99	1.00	0.99	179532
1	0.96	0.43	0.60	3748
accuracy			0.99	183280
macro avg	0.97	0.72	0.79	183280
weighted avg	0.99	0.99	0.99	183280

图 17 B 类信号的声发射精确度

由此可见准确度分别为 0.91 与 0.96，准确度较高，模型拟合较成功。通过随机森林，得出以下两表：

表 4 电磁辐射前兆特征时间区间

序号	时间区间起点	时间区间终点
1	2020-04-09 14:25:02	2020-04-09 21:57:15
2	2020-04-10 00:24:26	2020-04-10 10:38:22
3	2020-04-24 22:24:39	2020-04-24 23:44:04
4	2020-04-25 00:11:10	2020-04-25 00:47:00
5	2020-04-25 03:00:39	2020-04-25 03:05:29

表 5 声发射前兆特征时间区间

序号	时间区间起点	时间区间终点
1	2021-11-01 18:10:30	2021-11-01 18:14:08
2	2021-11-07 12:00:03	2021-11-07 12:19:59
3	2022-01-12 20:06:47	2022-01-12 20:21:25
4	2022-01-14 22:41:51	2022-01-14 23:25:44
5	2022-01-15 13:53:06	2022-01-15 19:16:56

5.3 问题三模型建立与求解

5.3.1 问题三求解思路

问题三同样运用小波分析对数据进行处理降噪后，对特征提取后的基因数据采用 BP 神经网络进行分类学习，训练获得分类网络模型，采用获得的分类网络模型对测试数进行分类，并对神经网络分类器的性能进行评估。

5.3.2 问题三模型建立

BP 神经网络算法的学习与训练流程由两个主要阶段构成：正向的信息流通和反向的误差调整。在正向阶段，输入的样本数据从起始的输入层开始，经过中间隐藏的层级，并通过各层的传递函数进行处理，最终到达输出层。然而，如果输出层的结果与预期的目标不符，就会启动反向阶段。在这个阶段，误差信号会沿着之前正向流通的路径逆向传回，基于均方误差和梯度下降的方法，对神经网络中的连接权重进行调整。这种调整的目的是使网络的实际输出与指导学习的目标信号之间的误差逐渐减小。这个过程会持续进行，直到达到预定的误差阈值或者达到了设定的最大训练次数。

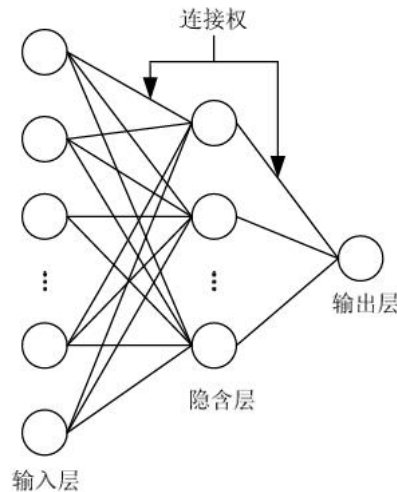


图 18 BP 神经网络拓扑结构

5.3.3 问题三模型求解与分析

设网络输入向量为 $X = (x_1, x_2, \dots, x_n)$, 实际输出为 $Y = (y_1, y_2, \dots, y_n)$ 期望的输出为 $TY = (ty_1, \dots, ty_n)$ 。TY 数据分为两类，0 表示正常，1 表示异常。给定隐藏层或输出层的神经元，其输入为 $I_j = \int_i O_i W_{ij} + \theta_{ij}$ ，其中 W_{ij} 是由上一层的神经元 i 到神经元 j 的链接权；传递函数 $O_i = \frac{1}{1+e^{-I_j}}$ 是神经元 i 的输出； θ_j 是神经元 j 的偏置。对应训练集中的第 k 个样

本，其误差函数为 $E = \frac{1}{2n} \sum_{j=1}^n (ty_{kj} - y_{kj})^2$ ，通常利用梯度下降法求误差函数的极小值，即 $\Delta W_{ij} = -\mu \frac{\partial E}{\partial W_{ij}}$ ，其中， μ 为学习速率，其值大于零。

通过对 AE 神经网络模型和 EMR 神经网络模型的准确度和损失度的计算与分析，得到下列两图：

```
Epoch [5/40], Loss: 1.2961
Epoch [10/40], Loss: 1.2316
Epoch [15/40], Loss: 1.1700
Epoch [20/40], Loss: 1.1103
Epoch [25/40], Loss: 1.0519
Epoch [30/40], Loss: 0.9942
Epoch [35/40], Loss: 0.9370
Epoch [40/40], Loss: 0.8802
Test Accuracy: 0.9681
```

图 19 AE 神经网络模型

通过该图可以 AE 神经网络模型的准确率为 0.96，且损失度不高于 1，由此可知该神经网络模型满足需求。

```
Epoch [5/40], Loss: 1.4341
Epoch [10/40], Loss: 1.3382
Epoch [15/40], Loss: 1.2479
Epoch [20/40], Loss: 1.1626
Epoch [25/40], Loss: 1.0819
Epoch [30/40], Loss: 1.0057
Epoch [35/40], Loss: 0.9342
Epoch [40/40], Loss: 0.8679
Test Accuracy: 0.8495
```

图 20 EMR 神经网络模型

通过该图可以 EMR 神经网络模型的准确率为 0.96，且损失度不高于 1，由此可知该神经网络模型满足需求。

那么根据模型可得出以下表格：

表 6 采集数据所在时刻出现前兆特征的概率

电磁辐射数据所在时刻	前兆特征的概率	声发射数据所在时刻	前兆特征的概率
2023-1-24 23:58:36	60.9%	2023-1-24 23:58:36	24%
2023-2-11 23:59:20	$4.8 \times 10^{-6}\%$	2023-2-11 23:59:20	$9.8 \times 10^{-5}\%$
2023-2-26 23:59:27	98.9%	2023-2-26 23:59:27	95.8%
2023-3-10 23:58:14	86.6%	2023-3-10 23:58:14	47.2%
2023-3-30 23:58:13	97.8%	2023-3-30 23:58:13	89.7%

六、模型评价与推广

6.1 模型的优点

- (1) 模型采用小波分析可以通过缩放小波母函数的宽度来获取信号的频率特征,同时,通过平移小波母函数来获得信号的时间信息。这使得小波分析在时频局部特性上表现出色,能够聚焦信号的任何细节。
- (2) 模型采用小波分析可以有效地去除信号中的噪声和干扰。通过将信号的高频部分进行平滑处理,小波分析可以消除噪声和干扰的影响,提高信号处理的准确性。
- (3) 模型采用随机森林方法,机森林对于大规模数据集有良好的处理能力,并且能够处理具有高度非线性关系的数据。它能够有效地处理高维数据,无需进行降维或特征选择。
- (4) 本文使用的小波分析算法,小波分析算法是一种新的变换分析方法,它继承和发展了短时傅立叶变换局部化的思想,同时又克服了窗口大小不随频率变化等缺点,能够提供一个随频率改变的“时间-频率”窗口,是进行信号时频分析和处理的理想工具。它的主要特点是通过变换能够充分突出问题某些方面的特征,能对时间(空间)频率的局部化分析,通过伸缩平移运算对信号(函数)逐步进行多尺度细化,最终达到高频处时间细分,低频处频率细分,能自动适应时频信号分析的要求,从而可聚焦到信号的任意细节,对此问题十分适用。
- (5) BP 神经网络可以处理各种类型的数据,包括数值型、分类型和序列型数据。同时,由于神经元的激活函数可以是非线性的,BP 神经网络可以处理非线性问题。

6.2 模型的不足

- (6) 实际应用中,传感器收集来源和煤矿的差异可能也是重要的因素,但本文未能考虑到这些因素的影响,一定程度上影响了模型的准确性。
- (7) 本文提出的模型对于现有条件使用效果较好,由于时间问题没有对其他情况进行检验。对于其他情形(如:模型的特征较多,数据处理较复杂的情况),可能无法达到较好的效果。
- (8) 本题中波的频率较低,小波分析在低频段,小波分析的时间分辨率较低,难以捕捉到信号的细节变化。
- (9) 模型没有考虑不同煤矿可能产生的不同影响,适用性受到一定的限制。
- (10) BP 神经网络需要大量的时间和数据来调整权重和阈值,以达到稳定的状态。这可能导致训练过程较长。

6.3 模型的推广

随机森林模型具有广泛的运用,可以用于多处现实生活中需要预测的场景。

① 金融行业中的信用评分:

金融机构通常需要对借款人进行信用评分,以评估其信用风险。随机森林模型可以通过分析借款人的个人和财务数据(如收入、债务、征信记录等)来预测借款人的违约概率。

推广随机森林模型时,可以将其应用于不同地区的金融市场或不同种类的贷款产品,以检验模型在不同场景下的泛化能力。

② 医疗领域中的疾病预测:

在医疗领域,随机森林模型可以用于预测患者的患病风险。例如,通过分析患者的生理指标、基因信息、生活习惯等数据,模型可以预测患者未来患某种疾病的可能性。

推广时,可以将模型应用于不同的疾病类型、不同的患者群体或不同的医疗机构,以评估模型在不同场景下的准确性和可靠性。

③ 交通运输中的交通流量预测:

随机森林模型在交通领域中也具有广泛的应用价值。例如，通过分析历史交通流量数据、天气条件、道路状况等因素，模型可以预测未来某个时间段内的交通流量情况。

推广时，可以将模型应用于不同的交通场景（如城市道路、高速公路、机场等），以检验模型在不同场景下的预测精度。

④ 电商网站的商品推荐：

电商网站通常会根据用户的购买历史和浏览行为向用户推荐相关的商品。随机森林模型可以根据用户的特征和行为数据来预测用户对某个商品的购买意向。

推广时，可以将模型应用于不同的电商网站、不同的商品类别或不同的用户群体，以检验模型在不同场景下的推荐效果。

⑤ 市场营销中的客户细分：

在市场营销中，随机森林模型可以用于客户细分，即将客户划分为不同的群体，以便为不同的群体提供个性化的营销策略。

推广时，可以将模型应用于不同的行业、不同的产品或不同的市场区域，以检验模型在不同市场环境下的有效性。

总而言之，随机森林模型在不同领域和场景中的推广应用，说明了其强大的泛化能力和广泛的适用性。当然，在推广过程中还需要根据具体场景进行模型参数的调整和优化，以确保模型能够在新场景下取得良好的性能。

参考文献

- [1] 王欣欣, 于师建 (山东科技大学矿业与安全工程学院, 山东 青岛 266590)
- [2] Machine Learning, 45, 5–32, 2001 c 2001 Kluwer Academic Publishers. Manufactured in The Netherlands.
- [3]

附录

附录 1

第一问、第二问中 EMR 的求解代码

```
EMR:
import pandas as pd
import numpy as np
import pywt

interference_data = pd.read_excel('附件 1EMR.xlsx') # 这里读入数据

def extract_wavelet_features(data, column_name=' 电 磁 辐 射 (EMR)',
scales=np.arange(1, 4), wavelet='cmor'):
    # 计算小波变换系数
    emr_values = data[column_name].values
    coefficients, frequencies = pywt.cwt(emr_values, scales, wavelet)
    # 计算各种小波特征
    scale_energies = np.sum(np.abs(coefficients) ** 2, axis=0)
    scale_means = np.mean(np.abs(coefficients), axis=0)
    scale_stds = np.std(np.abs(coefficients), axis=0)
    scale_max = np.max(np.abs(coefficients), axis=0)
    scale_min = np.min(np.abs(coefficients), axis=0)
    scale_median = np.median(np.abs(coefficients), axis=0)
    scale_ptp = np.ptp(np.abs(coefficients), axis=0)
    total_energy = np.sum(np.abs(coefficients) ** 2, axis=0) + 1e-10
    scale_energy_ratios = np.abs(coefficients) ** 2 / total_energy

    # 将所有特征汇总到一个特征矩阵中
    features = np.column_stack(
        [scale_energies, scale_means, scale_stds, scale_max, scale_min,
scale_median, scale_ptp] + [np.abs(coeff) for
coeff in
scale_energy_ratios])
    feature_names = ['energy', 'mean', 'std', 'max', 'min', 'median', 'ptp'] +
[f'energy_ratio_scale_{i + 1}' for i in
range(scale_energy_ratios.shape[0])]
    return pd.DataFrame(features, columns=feature_names)

def process_wavelet_features(data, column_name=' 电 磁 辐 射 (EMR)',
scales=np.arange(1, 4), wavelet='cmor'):
    # 提取小波特征
    wavelet_df = extract_wavelet_features(data, column_name, scales, wavelet)

    # 合并小波特征和原有 DataFrame
```

```

full_df = pd.concat([data, wavelet_df], axis=1)

# 创建时间基特征
full_df['时间 (time)'] = pd.to_datetime(full_df['时间 (time)'])
full_df['hour'] = full_df['时间 (time)'].dt.hour
full_df['day_of_week'] = full_df['时间 (time)'].dt.dayofweek
full_df['rolling_mean'] = full_df[column_name].rolling(window=10,
min_periods=1).mean()
full_df['rolling_max'] = full_df[column_name].rolling(window=10,
min_periods=1).max()
full_df['rolling_min'] = full_df[column_name].rolling(window=10,
min_periods=1).min()
return full_df

# 自定义的类别映射函数
def custom_class_mapping(label):
    return 1 if label == 'B' else 0

full_df = process_wavelet_features(interference_data)
# 应用自定义类别映射
full_df['类别 (class)'] = full_df['类别 (class)'].map(custom_class_mapping)

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report
from sklearn.preprocessing import StandardScaler

# 选择特征和标签
X = full_df.drop(['时间 (time)', '类别 (class)'], axis=1) # 去除时间和类别标签
y = full_df['类别 (class)']

# 划分训练集和测试集
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)

# 提取列名
feature_names = X_train.columns
# 标准化特征
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

from sklearn.ensemble import RandomForestClassifier
from sklearn.feature_selection import RFE
from tqdm import tqdm
import pandas as pd

```



```

# 自定义 RFE 类以支持 tqdm
class TqdmRFE(RFE):
    def fit(self, X, y):
        self._num_features = X.shape[1]
        with tqdm(total=self._num_features, desc="RFE Progress") as pbar:
            super().fit(X, y)
            for _ in range(self._num_features - self.n_features_to_select):
                pbar.update(1)

# 示例数据
# X_train 和 y_train 是已经存在的数据集

# 假设我们使用随机森林分类器
clf = RandomForestClassifier(n_estimators=15, random_state=42)

# 使用自定义的 TqdmRFE
rfe = TqdmRFE(estimator=clf, n_features_to_select=6, step=1)
rfe.fit(X_train, y_train)

# 查看选定的特征（布尔值）
selected_features = rfe.support_

# 查看特征的排名
feature_ranking = rfe.ranking_

# 创建一个 DataFrame 来显示每个特征的排名和是否被选中
feature_ranking_df = pd.DataFrame({
    'Feature Name': feature_names,
    'Selected': selected_features,
    'Ranking': feature_ranking
})

# 打印特征排名
print(feature_ranking_df.sort_values(by='Ranking'))

# 创建随机森林分类器模型
clf = RandomForestClassifier(n_estimators=15, random_state=42)
clf.fit(X_train, y_train)

# 预测测试集
y_pred = clf.predict(X_test)

# 评估模型
print(classification_report(y_test, y_pred))
# 读取测试数据并提取特征

```

```

test_data = pd.read_excel('附件 2EMR.xlsx', engine='openpyxl', sheet_name='时间
段 1')
full_test_df = process_wavelet_features(test_data)

# 准备测试集
X_test = full_test_df.drop(['时间 (time)'], axis=1)
X_test = scaler.transform(X_test)

# 预测测试集
predictions = clf.predict(X_test)

# 将预测结果和原始测试数据拼接
full_test_df['预测类别'] = predictions

# 找到预测类别为 1 的时间区间
class_1_df = full_test_df[full_test_df['预测类别'] == 1]

# 打印结果
print(class_1_df[['时间 (time)', '预测类别']])
# 读取测试数据并提取特征
test_data = pd.read_excel('附件 2EMR.xlsx', engine='openpyxl', sheet_name='时间
段 2')
full_test_df = process_wavelet_features(test_data)

# 准备测试集
X_test = full_test_df.drop(['时间 (time)'], axis=1)
X_test = scaler.transform(X_test)

# 预测测试集
predictions = clf.predict(X_test)

# 将预测结果和原始测试数据拼接
full_test_df['预测类别'] = predictions

# 找到预测类别为 1 的时间区间
class_1_df = full_test_df[full_test_df['预测类别'] == 1]

# 打印结果
print(class_1_df[['时间 (time)', '预测类别']])
# 读取测试数据并提取特征
test_data = pd.read_excel('附件 2EMR.xlsx', engine='openpyxl', sheet_name='时间
段 3')
full_test_df = process_wavelet_features(test_data)

# 准备测试集
X_test = full_test_df.drop(['时间 (time)'], axis=1)
X_test = scaler.transform(X_test)

```

```

# 预测测试集
predictions = clf.predict(X_test)

# 将预测结果和原始测试数据拼接
full_test_df['预测类别'] = predictions

# 找到预测类别为 1 的时间区间
class_1_df = full_test_df[full_test_df['预测类别'] == 1]

# 打印结果
print(class_1_df[['时间 (time)', '预测类别']])
np.save('my_array.npy', class_1_df)

```

附录 2

第一问、第二问中 AE 的求解代码

```

AE:
import pandas as pd
import numpy as np
import pywt

interference_data = pd.read_excel('附件 1AE.xlsx') # 这里读入数据

def extract_wavelet_features(data, column_name=' 声 波 强 度 (AE)',
scales=np.arange(1, 4), wavelet='cmor'):
    # 计算小波变换系数
    emr_values = data[column_name].values
    coefficients, frequencies = pywt.cwt(emr_values, scales, wavelet)
    # 计算各种小波特征
    scale_energies = np.sum(np.abs(coefficients) ** 2, axis=0)
    scale_means = np.mean(np.abs(coefficients), axis=0)
    scale_stds = np.std(np.abs(coefficients), axis=0)
    scale_max = np.max(np.abs(coefficients), axis=0)
    scale_min = np.min(np.abs(coefficients), axis=0)
    scale_median = np.median(np.abs(coefficients), axis=0)
    scale_ptp = np.ptp(np.abs(coefficients), axis=0)
    total_energy = np.sum(np.abs(coefficients) ** 2, axis=0) + 1e-10
    scale_energy_ratios = np.abs(coefficients) ** 2 / total_energy

    # 将所有特征汇总到一个特征矩阵中
    features = np.column_stack(
        [scale_energies, scale_means, scale_stds, scale_max, scale_min,
scale_median, scale_ptp] + [np.abs(coeff) for
coeff in

```

```

scale_energy_ratios])
    feature_names = ['energy', 'mean', 'std', 'max', 'min', 'median', 'ptp'] +
[f'energy_ratio_scale_{i + 1}' for i in
range(scale_energy_ratios.shape[0])]
    return pd.DataFrame(features, columns=feature_names)

def process_wavelet_features(data, column_name=' 声 波 强 度 (AE)',
scales=np.arange(1, 4), wavelet='cmor'):
    # 提取小波特征
    wavelet_df = extract_wavelet_features(data, column_name, scales, wavelet)

    # 合并小波特征和原有 DataFrame
    full_df = pd.concat([data, wavelet_df], axis=1)

    # 创建时间基特征
    full_df['时间 (time)'] = pd.to_datetime(full_df['时间 (time)'])
    full_df['hour'] = full_df['时间 (time)'].dt.hour
    full_df['day_of_week'] = full_df['时间 (time)'].dt.dayofweek
    full_df['rolling_mean'] = full_df[column_name].rolling(window=10,
min_periods=1).mean()
    full_df['rolling_max'] = full_df[column_name].rolling(window=10,
min_periods=1).max()
    full_df['rolling_min'] = full_df[column_name].rolling(window=10,
min_periods=1).min()

    return full_df

# 自定义的类别映射函数
def custom_class_mapping(label):
    return 1 if label == 'B' else 0

full_df = process_wavelet_features(interference_data)
# 应用自定义类别映射
full_df['类别 (class)'] = full_df['类别 (class)'].map(custom_class_mapping)

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report
from sklearn.preprocessing import StandardScaler

# 选择特征和标签
X = full_df.drop(['时间 (time)', '类别 (class)'], axis=1) # 去除时间和类别标签
y = full_df['类别 (class)']

```

```

# 划分训练集和测试集
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.1,
random_state=42)

# 提取列名
feature_names = X_train.columns
# 标准化特征
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

from sklearn.ensemble import RandomForestClassifier
from sklearn.feature_selection import RFE
from tqdm import tqdm
import pandas as pd

# 自定义 RFE 类以支持 tqdm
class TqdmRFE(RFE):
    def fit(self, X, y):
        self._num_features = X.shape[1]
        with tqdm(total=self._num_features, desc="RFE Progress") as pbar:
            super().fit(X, y)
            for _ in range(self._num_features - self.n_features_to_select):
                pbar.update(1)

# 示例数据
# X_train 和 y_train 是已经存在的数据集

# 假设我们使用随机森林分类器
clf = RandomForestClassifier(n_estimators=1, random_state=42)

# 使用自定义的 TqdmRFE
rfe = TqdmRFE(estimator=clf, n_features_to_select=6, step=1)
rfe.fit(X_train, y_train)

# 查看选定的特征（布尔值）
selected_features = rfe.support_

# 查看特征的排名
feature_ranking = rfe.ranking_

# 创建一个 DataFrame 来显示每个特征的排名和是否被选中
feature_ranking_df = pd.DataFrame({
    'Feature Name': feature_names,
    'Selected': selected_features,
    'Ranking': feature_ranking
})

```

```

# 打印特征排名
print(feature_ranking_df.sort_values(by='Ranking'))

# 创建随机森林分类器模型
clf = RandomForestClassifier(n_estimators=1, random_state=42)
clf.fit(X_train, y_train)

# 预测测试集
y_pred = clf.predict(X_test)

# 评估模型
print(classification_report(y_test, y_pred))
# 读取测试数据并提取特征
test_data = pd.read_excel('附件 2AE.xlsx', engine='openpyxl', sheet_name='时间段
1')
full_test_df = process_wavelet_features(test_data)

# 准备测试集
X_test = full_test_df.drop(['时间 (time)'], axis=1)
X_test = scaler.transform(X_test)

# 预测测试集
predictions = clf.predict(X_test)

# 将预测结果和原始测试数据拼接
full_test_df['预测类别'] = predictions

# 找到预测类别为 1 的时间区间
class_1_df = full_test_df[full_test_df['预测类别'] == 1]

# 打印结果
print(class_1_df[['时间 (time)', '预测类别']])
# 读取测试数据并提取特征
test_data = pd.read_excel('附件 2AE.xlsx', engine='openpyxl', sheet_name='时间段
2')
full_test_df = process_wavelet_features(test_data)

# 准备测试集
X_test = full_test_df.drop(['时间 (time)'], axis=1)
X_test = scaler.transform(X_test)

# 预测测试集
predictions = clf.predict(X_test)

# 将预测结果和原始测试数据拼接
full_test_df['预测类别'] = predictions

```

```

# 找到预测类别为 1 的时间区间
class_1_df = full_test_df[full_test_df['预测类别'] == 1]

# 打印结果
print(class_1_df[['时间 (time)', '预测类别']])
# 读取测试数据并提取特征
test_data = pd.read_excel('附件 2AE.xlsx', engine='openpyxl', sheet_name='时间段
3')
full_test_df = process_wavelet_features(test_data)

# 准备测试集
X_test = full_test_df.drop(['时间 (time)'], axis=1)
X_test = scaler.transform(X_test)

# 预测测试集
predictions = clf.predict(X_test)

# 将预测结果和原始测试数据拼接
full_test_df['预测类别'] = predictions

# 找到预测类别为 1 的时间区间
class_1_df = full_test_df[full_test_df['预测类别'] == 1]

# 打印结果
print(class_1_df[['时间 (time)', '预测类别']])

```

附录 3

第三问中 EMR 的求解代码

```

EMR:
import pandas as pd
import numpy as np
import pywt
train_data = pd.read_excel('附件 1EMR.xlsx') # 这里读入数据

def extract_wavelet_features(data, column_name='电磁辐射 (EMR)',
scales=np.arange(1, 4), wavelet='cmor'):
    # 计算小波变换系数
    emr_values = data[column_name].values
    coefficients, frequencies = pywt.cwt(emr_values, scales, wavelet)

    # 计算各种小波特征
    scale_energies = np.sum(np.abs(coefficients) ** 2, axis=0)
    scale_means = np.mean(np.abs(coefficients), axis=0)
    scale_stds = np.std(np.abs(coefficients), axis=0)

```

```

scale_max = np.max(np.abs(coefficients), axis=0)
scale_min = np.min(np.abs(coefficients), axis=0)
scale_median = np.median(np.abs(coefficients), axis=0)
scale_ptp = np.ptp(np.abs(coefficients), axis=0)
total_energy = np.sum(np.abs(coefficients) ** 2, axis=0) + 1e-10
scale_energy_ratios = np.abs(coefficients) ** 2 / total_energy

# 将所有特征汇总到一个特征矩阵中
features = np.column_stack(
    [scale_energies, scale_means, scale_stds, scale_max, scale_min,
scale_median, scale_ptp] + [np.abs(coeff) for
coeff in
scale_energy_ratios])
    feature_names = ['energy', 'mean', 'std', 'max', 'min', 'median', 'ptp'] +
[f'energy_ratio_scale_{i + 1}'] for i in
range(scale_energy_ratios.shape[0])]
    return pd.DataFrame(features, columns=feature_names)

def process_wavelet_features(data, column_name='电磁辐射 (EMR)',
scales=np.arange(1, 4), wavelet='cmor'):
    # 提取小波特征
    wavelet_df = extract_wavelet_features(data, column_name, scales, wavelet)

    # 计算时间间隔特征
    data['时间间隔 (interval)'] = data['时间
(time)'].diff().dt.total_seconds().fillna(0)

    # 合并小波特征和原有 DataFrame
    full_df = pd.concat([data, wavelet_df], axis=1)

    # 创建时间基特征
    full_df['时间 (time)'] = pd.to_datetime(full_df['时间 (time)'])
    full_df['hour'] = full_df['时间 (time)'].dt.hour
    full_df['day_of_week'] = full_df['时间 (time)'].dt.dayofweek
    full_df['rolling_mean'] = full_df[column_name].rolling(window=10,
min_periods=1).mean()
    full_df['rolling_max'] = full_df[column_name].rolling(window=10,
min_periods=1).max()
    full_df['rolling_min'] = full_df[column_name].rolling(window=10,
min_periods=1).min()

    return full_df
train_data = process_wavelet_features(train_data)
from sklearn.preprocessing import LabelEncoder

```



```

# 假设已经有一个 LabelEncoder 实例 le, 并且已经用它来转换了 full_df['类别 (class)']
# 转换类别标签为数值
le = LabelEncoder()
train_data['类别 (class)'] = le.fit_transform(train_data['类别 (class)'])

# 查看标签和编码的对应关系
label_mapping = dict(zip(le.classes_, range(len(le.classes_))))

# 打印每个标签及其对应的数字编码
for label, code in label_mapping.items():
    print(f'Label: {label} -> Code: {code}')
data = train_data
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import pandas as pd

# 1. 数据加载和预处理
# 假设 data 是一个 pandas DataFrame
features = data.drop(['类别 (class)', '时间 (time)'], axis=1).values
labels = data['类别 (class)'].values

# 数据标准化
scaler = StandardScaler()
features = scaler.fit_transform(features)

# 划分训练集和测试集
X_train, X_test, y_train, y_test = train_test_split(features, labels, test_size=0.1,
random_state=42)

# 转换为 PyTorch 张量
X_train = torch.tensor(X_train, dtype=torch.float32)
X_test = torch.tensor(X_test, dtype=torch.float32)
y_train = torch.tensor(y_train, dtype=torch.long)
y_test = torch.tensor(y_test, dtype=torch.long)

# 2. 构建模型
class BPNetwork(nn.Module):
    def __init__(self, input_dim, hidden_dim, output_dim):
        super(BPNetwork, self).__init__()
        self.fc1 = nn.Linear(input_dim, hidden_dim)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(hidden_dim, output_dim)
        self.log_softmax = nn.LogSoftmax(dim=1) # 使用 log-softmax

```

```

def forward(self, x):
    x = self.fc1(x)
    x = self.relu(x)
    x = self.fc2(x)
    x = self.log_softmax(x) # 使用 log-softmax 代替 softmax
    return x

input_dim = X_train.shape[1]
hidden_dim = 32 # 可以根据需要调整
output_dim = len(set(labels)) # 类别数量
model = BPNetwork(input_dim, hidden_dim, output_dim)

# 3. 定义损失函数和优化器
criterion = nn.NLLLoss() # 使用 NLLLoss
optimizer = optim.Adam(model.parameters(), lr=0.002)

# 4. 训练模型
num_epochs = 60 # 可以根据需要调整
for epoch in range(num_epochs):
    optimizer.zero_grad()
    outputs = model(X_train)
    loss = criterion(outputs, y_train)
    loss.backward()
    optimizer.step()
    if (epoch + 1) % 5 == 0:
        print(f'Epoch [{epoch+1}/{num_epochs}], Loss: {loss.item():.4f}')

# 5. 评估模型
model.eval()
with torch.no_grad():
    outputs = model(X_test)
    probabilities = torch.exp(outputs) # 通过取指数获得概率
    predicted_labels = torch.argmax(probabilities, dim=1)
    accuracy = (predicted_labels == y_test).sum().item() / len(y_test)
    print(f'Test Accuracy: {accuracy:.4f}')
# # 打印每个样本的类别概率
# for i, probs in enumerate(probabilities):
#     print(f'Sample {i}: {probs.numpy()}')
# 假设模型已被训练并保存在变量 model 中
model.eval()

# 读取测试数据并提取特征
test_data = pd.read_excel('附件 3EMR.xlsx', engine='openpyxl', sheet_name='时间段 1')
full_test_df = process_wavelet_features(test_data)

# 准备测试集
X_test = full_test_df.drop(['时间 (time)'], axis=1).values

```

```

X_test = torch.tensor(X_test, dtype=torch.float32)

# 预测测试集
with torch.no_grad():
    output_probabilities = torch.exp(model(X_test))

# 获取最后一个时间点的概率
last_time_prob = output_probabilities[-1]

# 输出最后一个时间点的预测概率
print(f'最后一个时间点的预测概率: {last_time_prob.numpy()}')
# 假设模型已被训练并保存在变量 model 中
model.eval()

# 读取测试数据并提取特征
test_data = pd.read_excel('附件 3EMR.xlsx', engine='openpyxl', sheet_name='时间
段 2')
full_test_df = process_wavelet_features(test_data)

# 准备测试集
X_test = full_test_df.drop(['时间 (time)'], axis=1).values
X_test = torch.tensor(X_test, dtype=torch.float32)

# 预测测试集
with torch.no_grad():
    output_probabilities = torch.exp(model(X_test))

# 获取最后一个时间点的概率
last_time_prob = output_probabilities[-1]

# 输出最后一个时间点的预测概率
print(f'最后一个时间点的预测概率: {last_time_prob.numpy()}')
# 假设模型已被训练并保存在变量 model 中
model.eval()

# 读取测试数据并提取特征
test_data = pd.read_excel('附件 3EMR.xlsx', engine='openpyxl', sheet_name='时间
段 3')
full_test_df = process_wavelet_features(test_data)

# 准备测试集
X_test = full_test_df.drop(['时间 (time)'], axis=1).values
X_test = torch.tensor(X_test, dtype=torch.float32)

# 预测测试集
with torch.no_grad():
    output_probabilities = torch.exp(model(X_test))

```

```

# 获取最后一个时间点的概率
last_time_prob = output_probabilities[-1]

# 输出最后一个时间点的预测概率
print(f'最后一个时间点的预测概率: {last_time_prob.numpy()}')
# 假设模型已被训练并保存在变量 model 中
model.eval()

# 读取测试数据并提取特征
test_data = pd.read_excel('附件 3EMR.xlsx', engine='openpyxl', sheet_name='时间
段 4')
full_test_df = process_wavelet_features(test_data)

# 准备测试集
X_test = full_test_df.drop(['时间 (time)'], axis=1).values
X_test = torch.tensor(X_test, dtype=torch.float32)

# 预测测试集
with torch.no_grad():
    output_probabilities = torch.exp(model(X_test))

# 获取最后一个时间点的概率
last_time_prob = output_probabilities[-1]

# 输出最后一个时间点的预测概率
print(f'最后一个时间点的预测概率: {last_time_prob.numpy()}')
# 假设模型已被训练并保存在变量 model 中
model.eval()

# 读取测试数据并提取特征
test_data = pd.read_excel('附件 3EMR.xlsx', engine='openpyxl', sheet_name='时间
段 5')
full_test_df = process_wavelet_features(test_data)

# 准备测试集
X_test = full_test_df.drop(['时间 (time)'], axis=1).values
X_test = torch.tensor(X_test, dtype=torch.float32)

# 预测测试集
with torch.no_grad():
    output_probabilities = torch.exp(model(X_test))

# 获取最后一个时间点的概率
last_time_prob = output_probabilities[-1]

# 输出最后一个时间点的预测概率

```

```
print(f'最后一个时间点的预测概率: {last_time_prob.numpy()}')
```

附录 4

第三问中 AE 的求解代码

```
AE:
import pandas as pd
import numpy as np
import pywt
train_data = pd.read_excel('附件 1AE.xlsx') # 这里读入数据

def extract_wavelet_features(data, column_name=' 声 波 强 度 (AE)',
scales=np.arange(1, 4), wavelet='cmor'):
    # 计算小波变换系数
    emr_values = data[column_name].values
    coefficients, frequencies = pywt.cwt(emr_values, scales, wavelet)

    # 计算各种小波特征
    scale_energies = np.sum(np.abs(coefficients) ** 2, axis=0)
    scale_means = np.mean(np.abs(coefficients), axis=0)
    scale_stds = np.std(np.abs(coefficients), axis=0)
    scale_max = np.max(np.abs(coefficients), axis=0)
    scale_min = np.min(np.abs(coefficients), axis=0)
    scale_median = np.median(np.abs(coefficients), axis=0)
    scale_ptp = np.ptp(np.abs(coefficients), axis=0)
    total_energy = np.sum(np.abs(coefficients) ** 2, axis=0) + 1e-10
    scale_energy_ratios = np.abs(coefficients) ** 2 / total_energy

    # 将所有特征汇总到一个特征矩阵中
    features = np.column_stack(
        [scale_energies, scale_means, scale_stds, scale_max, scale_min,
scale_median, scale_ptp] + [np.abs(coeff) for
coeff in
scale_energy_ratios])
    feature_names = ['energy', 'mean', 'std', 'max', 'min', 'median', 'ptp'] +
[f'energy_ratio_scale_{i + 1}' for i in
range(scale_energy_ratios.shape[0])]
    return pd.DataFrame(features, columns=feature_names)

def process_wavelet_features(data, column_name=' 声 波 强 度 (AE)',
scales=np.arange(1, 4), wavelet='cmor'):
    # 提取小波特征
    wavelet_df = extract_wavelet_features(data, column_name, scales, wavelet)
```

```

        # 计算时间间隔特征
        data['时间间隔 (interval)'] = data['时间 (time)'].diff().dt.total_seconds().fillna(0)

    # 合并小波特征和原有 DataFrame
    full_df = pd.concat([data, wavelet_df], axis=1)

    # 创建时间基特征
    full_df['时间 (time)'] = pd.to_datetime(full_df['时间 (time)'])
    full_df['hour'] = full_df['时间 (time)'].dt.hour
    full_df['day_of_week'] = full_df['时间 (time)'].dt.dayofweek
    full_df['rolling_mean'] = full_df[column_name].rolling(window=10,
min_periods=1).mean()
    full_df['rolling_max'] = full_df[column_name].rolling(window=10,
min_periods=1).max()
    full_df['rolling_min'] = full_df[column_name].rolling(window=10,
min_periods=1).min()

    return full_df
train_data = process_wavelet_features(train_data)
from sklearn.preprocessing import LabelEncoder
# 假设已经有一个 LabelEncoder 实例 le, 并且已经用它来转换了 full_df['类别 (class)']
# 转换类别标签为数值
le = LabelEncoder()
train_data['类别 (class)'] = le.fit_transform(train_data['类别 (class)'])

# 查看标签和编码的对应关系
label_mapping = dict(zip(le.classes_, range(len(le.classes_))))

# 打印每个标签及其对应的数字编码
for label, code in label_mapping.items():
    print(f'Label: {label} -> Code: {code}')
data = train_data
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import pandas as pd

# 1. 数据加载和预处理
# 假设 data 是一个 pandas DataFrame
features = data.drop(['类别 (class)', '时间 (time)'], axis=1).values
labels = data['类别 (class)'].values

# 数据标准化
scaler = StandardScaler()

```

```

features = scaler.fit_transform(features)

# 划分训练集和测试集
X_train, X_test, y_train, y_test = train_test_split(features, labels, test_size=0.1,
random_state=42)

# 转换为 PyTorch 张量
X_train = torch.tensor(X_train, dtype=torch.float32)
X_test = torch.tensor(X_test, dtype=torch.float32)
y_train = torch.tensor(y_train, dtype=torch.long)
y_test = torch.tensor(y_test, dtype=torch.long)

# 2. 构建模型
class BPNetwork(nn.Module):
    def __init__(self, input_dim, hidden_dim, output_dim):
        super(BPNetwork, self).__init__()
        self.fc1 = nn.Linear(input_dim, hidden_dim)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(hidden_dim, output_dim)
        self.log_softmax = nn.LogSoftmax(dim=1) # 使用 log-softmax

    def forward(self, x):
        x = self.fc1(x)
        x = self.relu(x)
        x = self.fc2(x)
        x = self.log_softmax(x) # 使用 log-softmax 代替 softmax
        return x

input_dim = X_train.shape[1]
hidden_dim = 64 # 可以根据需要调整
output_dim = len(set(labels)) # 类别数量
model = BPNetwork(input_dim, hidden_dim, output_dim)

# 3. 定义损失函数和优化器
criterion = nn.NLLLoss() # 使用 NLLLoss
optimizer = optim.Adam(model.parameters(), lr=0.001)

# 4. 训练模型
num_epochs = 40 # 可以根据需要调整
for epoch in range(num_epochs):
    optimizer.zero_grad()
    outputs = model(X_train)
    loss = criterion(outputs, y_train)
    loss.backward()
    optimizer.step()

    if (epoch + 1) % 5 == 0:
        print(f'Epoch [{epoch+1}/{num_epochs}], Loss: {loss.item():.4f}')

```

```

# 5. 评估模型
model.eval()
with torch.no_grad():
    outputs = model(X_test)
    probabilities = torch.exp(outputs) # 通过取指数获得概率
    predicted_labels = torch.argmax(probabilities, dim=1)
    accuracy = (predicted_labels == y_test).sum().item() / len(y_test)
    print(f'Test Accuracy: {accuracy:.4f}')
# # 打印每个样本的类别概率
# for i, probs in enumerate(probabilities):
#     print(f'Sample {i}: {probs.numpy()}')
# 假设模型已被训练并保存在变量 model 中
model.eval()

# 读取测试数据并提取特征
test_data = pd.read_excel('附件 3AE.xlsx', engine='openpyxl', sheet_name='时间段
1')
full_test_df = process_wavelet_features(test_data)

# 准备测试集
X_test = full_test_df.drop(['时间 (time)'], axis=1).values
X_test = torch.tensor(X_test, dtype=torch.float32)

# 预测测试集
with torch.no_grad():
    output_probabilities = torch.exp(model(X_test))

# 获取最后一个时间点的概率
last_time_prob = output_probabilities[-1]

# 输出最后一个时间点的预测概率
print(f'最后一个时间点的预测概率: {last_time_prob.numpy()}')
# 假设模型已被训练并保存在变量 model 中
model.eval()

# 读取测试数据并提取特征
test_data = pd.read_excel('附件 3AE.xlsx', engine='openpyxl', sheet_name='时间段
2')
full_test_df = process_wavelet_features(test_data)

# 准备测试集
X_test = full_test_df.drop(['时间 (time)'], axis=1).values
X_test = torch.tensor(X_test, dtype=torch.float32)

# 预测测试集
with torch.no_grad():
    output_probabilities = torch.exp(model(X_test))

```



```

# 获取最后一个时间点的概率
last_time_prob = output_probabilities[-1]

# 输出最后一个时间点的预测概率
print(f'最后一个时间点的预测概率: {last_time_prob.numpy()}')
# 假设模型已被训练并保存在变量 model 中
model.eval()

# 读取测试数据并提取特征
test_data = pd.read_excel('附件 3AE.xlsx', engine='openpyxl', sheet_name='时间段
3')
full_test_df = process_wavelet_features(test_data)

# 准备测试集
X_test = full_test_df.drop(['时间 (time)'], axis=1).values
X_test = torch.tensor(X_test, dtype=torch.float32)

# 预测测试集
with torch.no_grad():
    output_probabilities = torch.exp(model(X_test))

# 获取最后一个时间点的概率
last_time_prob = output_probabilities[-1]

# 输出最后一个时间点的预测概率
print(f'最后一个时间点的预测概率: {last_time_prob.numpy()}')
# 假设模型已被训练并保存在变量 model 中
model.eval()

# 读取测试数据并提取特征
test_data = pd.read_excel('附件 3AE.xlsx', engine='openpyxl', sheet_name='时间段
4')
full_test_df = process_wavelet_features(test_data)

# 准备测试集
X_test = full_test_df.drop(['时间 (time)'], axis=1).values
X_test = torch.tensor(X_test, dtype=torch.float32)

# 预测测试集
with torch.no_grad():
    output_probabilities = torch.exp(model(X_test))

# 获取最后一个时间点的概率
last_time_prob = output_probabilities[-1]

# 输出最后一个时间点的预测概率
print(f'最后一个时间点的预测概率: {last_time_prob.numpy()}')

```

```

# 假设模型已被训练并保存在变量 model 中
model.eval()

# 读取测试数据并提取特征
test_data = pd.read_excel('附件 3AE.xlsx', engine='openpyxl', sheet_name='时间段
5')
full_test_df = process_wavelet_features(test_data)

# 准备测试集
X_test = full_test_df.drop(['时间 (time)'], axis=1).values
X_test = torch.tensor(X_test, dtype=torch.float32)

# 预测测试集
with torch.no_grad():
    output_probabilities = torch.exp(model(X_test))

# 获取最后一个时间点的概率
last_time_prob = output_probabilities[-1]

# 输出最后一个时间点的预测概率
print(f'最后一个时间点的预测概率: {last_time_prob.numpy()}')

```